

Major Project Report

Full Stack Todo Application

Submitted in partial fulfillment of the requirement for the award of Internship
in

InternsElite

Submitted BY:

Tushar Saini(Roll No. WD-FSWD-A4/JAN-8929)

Under the Guidance of

Mr. Pratham Saxena

Designation



InternsElite Edutech Private Limited

Acknowledgement

I would like to express my sincere gratitude to my instructor/guide for their valuable guidance, support, and encouragement throughout the development of this project titled "Full Stack Todo Application." Their continuous feedback and insights helped me understand the concepts of full-stack development more effectively.

I would also like to thank my institution for providing the necessary resources and environment to successfully complete this project.

I am grateful to the developers and communities behind technologies such as Node.js, Express.js, and MongoDB, which made it possible to build this application efficiently.

Lastly, I would like to thank my friends and peers for their support and motivation during the project.

Name: Tushar saini

Project Title and Objective

Title: Full Stack Todo Application with API & Database

Objective: The goal is to build a robust Full Stack Todo Application. Students must develop a frontend interface that communicates with a custom-built backend API. This API will handle the business logic and interface with a persistent database, ensuring that task data is securely stored and globally accessible via authenticated requests.

1. Project Description

The **Full Stack Todo Application** is a web-based task management system designed to help users efficiently organize and manage their daily activities. This application allows users to create, read, update, and delete tasks (CRUD operations) through a simple and interactive user interface.

The project follows a **client-server architecture**, where the frontend handles user interaction and the backend manages business logic and data processing. The frontend communicates with the backend through RESTful APIs, ensuring smooth data exchange and real-time updates.

The backend of the application is developed using Node.js and Express.js, which handle API requests, routing, and server-side logic. The data is stored in MongoDB, a NoSQL database that efficiently manages task-related information such as task title and completion status.

Users can add new tasks, mark tasks as completed, edit existing tasks, and delete tasks when they are no longer needed. The application ensures that all data is stored persistently, allowing users to access their tasks even after refreshing or restarting the application.

This project demonstrates core full-stack development concepts such as API development, database integration, asynchronous operations, and dynamic user interface updates. It serves as a practical example of how modern web applications are built using a combination of frontend and backend technologies.

2. Technologies Used

Frontend Technologies

- **HTML (HyperText Markup Language):** Used to structure the web pages.
- **CSS (Cascading Style Sheets):** Used for designing and styling the user interface.

- **JavaScript:** Handles user interactions and dynamic content updates.
- **React:** Used to build a responsive and component-based user interface.

Backend Technologies

- **Node.js:** Provides the runtime environment to execute server-side JavaScript.
- **Express.js:** Used to build RESTful APIs and handle routing.

Database

- **MongoDB:** A NoSQL database used to store task data in a flexible JSON-like format.
- **Mongoose:** An Object Data Modeling (ODM) library used to interact with MongoDB

Other Tools & Libraries

- **dotenv:** Manages environment variables securely.
- **cors:** Enables communication between frontend and backend.
- **nodemon:** Automatically restarts the server during development.
- **Git & GitHub:** Used for version control and project hosting.

3. Setup Instructions

For Backend :

```
cd backend
```

```
npm install  
npm run dev
```

For Frontend

```
cd frontend  
npm install  
npm run dev
```

4. API Endpoints

Auth Endpoints:

Method	Endpoint	Body	Response
POST	<code>/api/auth/register</code>	<code>{username, email, password}</code>	<code>{token, user}</code>

POST	<code>/api/auth/login</code>	<code>{email, password}</code>	<code>{token, user}</code>
GET	<code>/api/auth/me</code>	—	<code>{user}</code>

Todo end points:

Method	Endpoint	Params / Body	Response
GET	<code>/api/todos</code>	<code>?completed & priority & search & sortBy & sortDir</code>	<code>{todos, count}</code>
GET	<code>/api/todos/stats</code>	—	<code>{total, completed, pending, byPriority}</code>
GET	<code>/api/todos/:id</code>	—	<code>{todo}</code>
POST	<code>/api/todos</code>	<code>{title, description?, priority?, dueDate?}</code>	<code>{todo}</code>
PATCH	<code>/api/todos/:id</code>	any todo fields	<code>{todo}</code>
DELETE	<code>/api/todos/:id</code>	—	<code>{message}</code>
DELETE	<code>/api/todos</code>	—	Bulk delete completed

Conclusion

The **Full Stack Todo Application** successfully demonstrates the development of a complete web application using modern technologies. The project effectively implements CRUD operations, allowing users to create, view, update, and delete tasks in an efficient and user-friendly manner.

Through this project, important concepts of full-stack development were understood, including frontend design, backend API development, and database integration. The use of technologies like Node.js, Express.js, and MongoDB helped in building a scalable and efficient application.

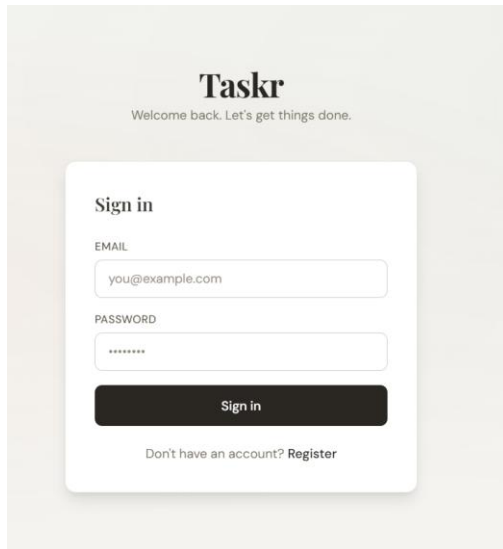
Additionally, the project provided practical experience in handling asynchronous operations, managing data flow between client and server, and designing a responsive user interface. It

also helped in improving problem-solving skills and understanding real-world application architecture.

In conclusion, this project serves as a strong foundation for developing more advanced web applications and enhances the overall understanding of full-stack development.

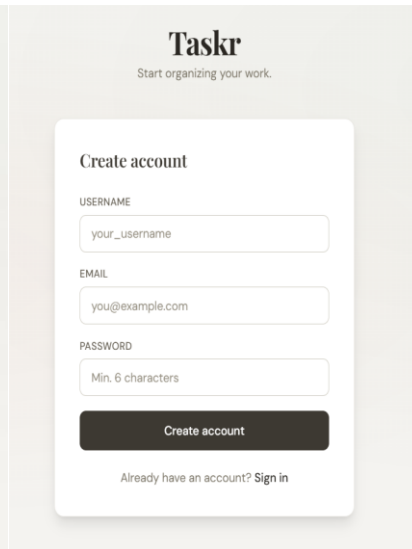
Result:

Sign in page



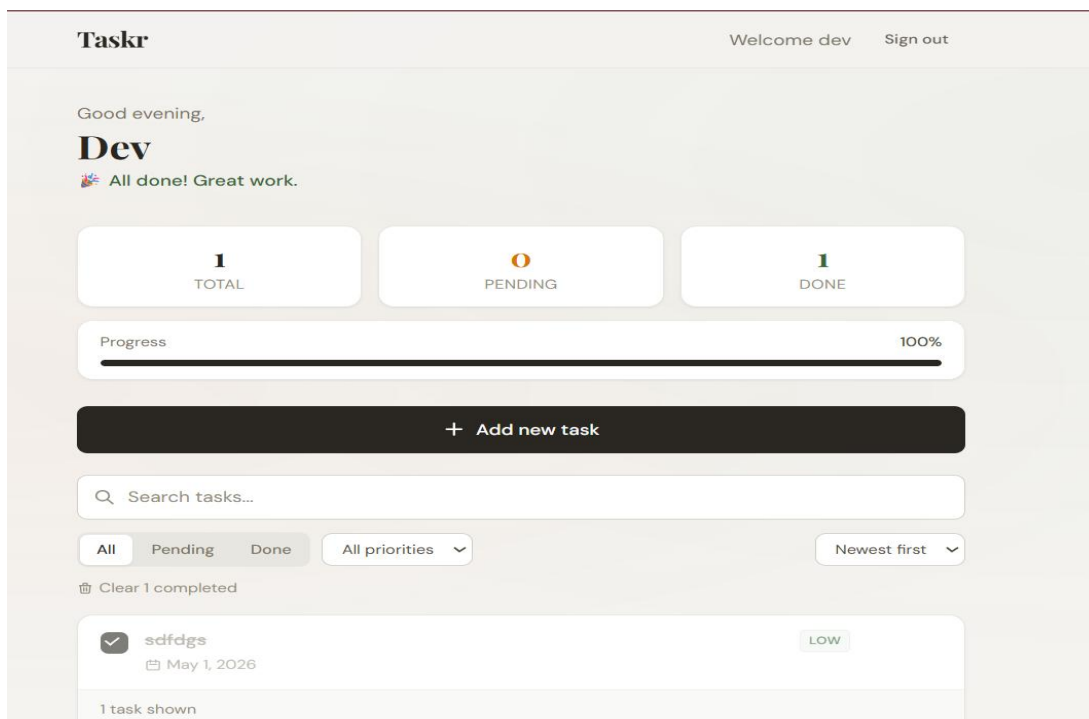
The sign-in page for Taskr features a light gray background. At the top, the 'Taskr' logo is displayed in a large, bold, black serif font, with the tagline 'Welcome back. Let's get things done.' in a smaller, lighter gray font below it. The main content is a white card with a subtle drop shadow. Inside the card, the heading 'Sign in' is followed by three input fields: 'EMAIL' (containing 'you@example.com'), 'PASSWORD' (containing '*****'), and a 'Sign in' button. At the bottom of the card, there is a link that says 'Don't have an account? Register'.

Sign up page



The sign-up page for Taskr has a light gray background. At the top, the 'Taskr' logo is in a large, bold, black serif font, with the tagline 'Start organizing your work.' in a smaller, lighter gray font below it. The main content is a white card with a subtle drop shadow. Inside the card, the heading 'Create account' is followed by three input fields: 'USERNAME' (containing 'your_username'), 'EMAIL' (containing 'you@example.com'), and 'PASSWORD' (containing 'Min, 6 characters'). Below these fields is a 'Create account' button. At the bottom of the card, there is a link that says 'Already have an account? Sign in'.

Dashboard



The dashboard for Taskr features a light gray background. At the top, the 'Taskr' logo is on the left, and 'Welcome dev' and 'Sign out' are on the right. Below the header, there is a greeting 'Good evening, Dev' and a message '🎉 All done! Great work.' The main content area contains three white cards with rounded corners: '1 TOTAL', '0 PENDING', and '1 DONE'. Below these cards is a progress bar labeled 'Progress' that is 100% full. A dark gray button with a white plus icon and the text '+ Add new task' is positioned below the progress bar. A search bar with the placeholder 'Search tasks...' is located below the button. Below the search bar are three tabs: 'All', 'Pending', and 'Done', with 'All' selected. To the right of the tabs is a dropdown menu labeled 'All priorities' with a downward arrow. Below the tabs and dropdown is a link that says 'Clear 1 completed'. At the bottom, there is a task card with a checkmark icon, the text 'sdfds', a date icon and 'May 1, 2026', and a 'LOW' priority label. The task card is followed by a link that says '1 task shown'.

Add new task

New task

TASK TITLE *

What needs to be done?

DESCRIPTION

Add more details...

PRIORITY

Low

Medium

High

DUE DATE & TIME

dd-mm-yyyy

--:--

Cancel

Create task

Taskr

Welcome dev

Sign out

You have 1 task pending

2 TOTAL

1 PENDING

1 DONE

Progress

50%

+ Add new task

Search tasks...

All Pending Done

All priorities

Newest first

Clear 1 completed

☐ study

May 2, 2026

HIGH

☒ sdfdgs

May 1, 2026

LOW

2 tasks shown